$_

MiniFS

Système de Fichiers Minimal avec FUSE

Version **1.0.0** | Langage **C (C11)** | Plateforme **Linux / FUSE3**

Rapport de Réalisation — Mars 2025

Sommaire

Au programme de cette présentation

01 Introduction

02 Analyse de l'existant

03 Architecture de la solution

04 Choix technologiques

05 Environnement de développement

06 Implémentation technique

07 Démonstration

08 Conclusion

Introduction

Qu'est-ce que MiniFS ?

MiniFS en résumé

MiniFS est un système de fichiers minimal éducatif implémenté avec FUSE (Filesystem in Userspace) pour Linux.

Il illustre concrètement les mécanismes internes d'un OS :

- Gestion des inodes et bitmaps d'allocation
- Lecture / écriture de blocs de 4 Ko
- Navigation dans les répertoires POSIX
- Montage transparent via le VFS Linux
- Outils complets : mkfs, fsck, debugfs

Spécifications

Taille de bloc	4 096 o
Max inodes	4 096
Max fichier	~50 Mio
Format	Little-endian
Licence	MIT
Version	1.0.0

Analyse de l'Existant

Comparaison des systèmes de fichiers

FS	Bloc	Max fichier	Inodes	Journalisation	Usage
ext4	4 Ko	16 Tio	Dynamiques	✓	Production Linux
FAT32	32 Ko	4 Gio	Aucun (FAT)	✗	Compatibilité universelle
NTFS	64 Ko	16 Eio	MFT	✓	Windows
MiniFS	**4 Ko**	**~50 Mio**	**4 096**	**✗**	**Éducatif / FUSE**

Pourquoi FUSE ?

FUSE (Filesystem in Userspace) permet d'implémenter un FS en espace utilisateur, sans risque de kernel panic. Le noyau Linux délègue les appels VFS au démon via libfuse3, offrant développement simplifié, débogage facilité, et sécurité accrue — au prix d'une légère surcharge de contexte.

Architecture — Disposition On-Disk

Organisation physique des données sur le disque

SUPERBLOC	BITMAP INODES	TABLE DES INODES	BITMAP BLOCS	BLOCS DE DONNÉES
Bloc 0	Bloc 1	Blocs 2–N	Blocs N+1–M	Blocs M+1–FIN
Magic, version, état, compteurs, offsets	1 bit / inode (0=libre, 1=occupé)	256 octets par inode (mode, taille, pointeurs…)	1 bit par bloc de données	Contenu des fichiers et répertoires

Structure d'un Inode (256 octets)

- mode (16b) • uid / gid (32b)
- size (64b) • links_count (16b)
- atime / mtime / ctime (64b each)
- 12 pointeurs directs (64b each)
- 1 pointeur indirect (64b)
- blocks_count + flags + reserved

Entrée de Répertoire

- inode (32b) • rec_len (16b)
- name_len (8b) • file_type (8b)
- name[256] — jusqu'à 255 caractères

Types : REG, DIR, CHRDEV, BLKDEV, FIFO, SOCK, SYMLINK

Architecture — Pile Logicielle

De l'appel utilisateur au disque

Appels système

- `open()`
- `read()`
- `write()`
- `stat()`
- `mkdir()`
- `unlink()`

Applications Utilisateur

shell, cp, cat, éditeurs…

VFS

open(), read(), write(), stat()…

Interface FUSE (libfuse3)

Module fuse.ko — callbacks en espace utilisateur

MiniFS (minifs_fuse)

getattr, readdir, create, read, write, unlink…

Image Disque / Périphérique Bloc

pread() / pwrite() positionnés — I/O direct

Callbacks FUSE

- `.getattr`
- `.readdir`
- `.create`
- `.read`
- `.write`
- `.unlink`

Choix Technologiques & Justification

Langage

C11

Contrôle précis sur la mémoire, stdint.h pour les types on-disk, API FUSE native en C

Interface

FUSE 3

API modernisée (fuse_config), espace utilisateur → zéro risque kernel, débogage simplifié

Concurrence

pthreads

Mutex POSIX standard protégeant les bitmaps et superbloc contre les accès concurrents

Build

GCC / Make

Détection auto de FUSE via pkg-config, cibles release (-O2) et debug (-g -O0)

Debug

Valgrind

--leak-check=full valide la libération correcte des bitmaps et buffers de blocs

Traçage

strace

Vérifie les offsets exacts des pread/pwrite sur l'image disque en temps réel

Environnement de Développement

```bash
# OpenSUSE Tumbleweed
$ sudo zypper install -y \
    gcc make fuse3 fuse3-devel \
    pkg-config git valgrind

# Compilation
$ make clean && make
$ make DEBUG=1   # mode debug
$ sudo make install
```

GDB

Débogage interactif avec breakpoints

Valgrind

Détection fuites mémoire

strace

Traçage appels système

debugfs

Inspection image disque MiniFS

Structure du projet

`src/`	8 modules C : main, filesystem, inode, block, directory, file_ops, utils, debug
`include/`	minifs.h — toutes les structures, constantes, prototypes
`tools/`	mkfs.c fsck.c debugfs.c mount_helper.c
`tests/`	test_suite.sh — suite de tests fonctionnels bash

Implémentation Technique

Les 6 modules C au cœur de MiniFS

`filesystem.c` — *Format & montage*

minifs_format(), minifs_mount(), minifs_sync() — calcul de la disposition, init des structures, mutex

`inode.c` — *Gestion des inodes*

alloc/free/read/write inode, résolution bloc direct/indirect via minifs_get_block()

`block.c` — *Allocation de blocs*

alloc/free/read/write bloc, initialisation à zéro (sécurité), protection bitmap sous verrou

`directory.c` — *Opérations répertoire*

readdir, lookup (strncmp), add/remove dir_entry, extension de bloc si nécessaire

`file_ops.c` — *Opérations fichiers*

create (alloc inode+dir entry), read (pagination), write (on-demand allocation), unlink

`main.c` — *Driver FUSE*

Table des callbacks fuse_operations, init du contexte global, lancement de fuse_main()

Démonstration — Mise en Route

Créer, formater et monter une image MiniFS

1 Créer l'image

```bash
# Image de 100 Mo
$ dd if=/dev/zero \
    of=disk.img \
    bs=1M count=100
```

2 Formater

```bash
# Formatage MiniFS
$ mkfs.minifs disk.img

Filesystem created successfully.
```

3 Monter

```bash
# Montage
$ mkdir -p /mnt/minifs
$ sudo mount.minifs \
    disk.img /mnt/minifs
```

Sortie de mkfs.minifs :

```
Block size: 4096 bytes    Total blocks: 25600 (100.0 MB)    Inode count: 4096

  Superblock:     1 block  (0)          Inode bitmap:   1 block  (1)

  Inode table:  256 blocks (2-257)       Data blocks: 25335 blocks (265-25599)
```

Démonstration — Utilisation & Vérification

Opérations fichiers en temps réel

bash

```bash
# Créer fichiers et répertoires
$ echo 'Hello MiniFS!' > /mnt/minifs/test.txt
$ mkdir /mnt/minifs/documents
$ cp rapport.pdf /mnt/minifs/documents/

# Lister le contenu
$ ls -la /mnt/minifs/
total 8
drwxr-xr-x  2 user user 4096 mars  1 test.txt
drwxr-xr-x  2 user user 4096 mars  1 documents/
```

bash

```bash
# Vérification intégrité
$ sudo fsck.minifs disk.img
Checking superblock... OK
Checking inode table... OK
Checking block bitmap... OK
No errors found.

# Statistiques
$ sudo debugfs.minifs disk.img --stats
Free blocks: 25332 / 25335
```

bash

```bash
# Démontage propre
$ sudo umount /mnt/minifs
# ou
$ sudo fusermount3 -u /mnt/minifs
```

bash

```bash
# Test de performance
$ for i in {1..100}; do
    echo "File $i" > file_$i.txt
  done
$ time cat file_*.txt > /dev/null
```

Conclusion

Système fonctionnel

MiniFS peut être monté, utilisé et démonté sur toute distribution Linux — toutes les opérations POSIX de base sont opérationnelles.

Architecture pédagogique

6 modules C indépendants illustrent clairement chaque couche : superbloc, inodes, blocs, répertoires, fichiers, driver FUSE.

Chaîne de build complète

Makefile avec détection auto de FUSE, modes debug/release, installation système et suite de tests automatisés.